

位运算

1. 常用内建函数

`__gcd()` // $O(\log \min(a,b))$

`__lg()` = `32 - __builtin_clz()` - 1 // 二进制对数 (也可表示最高位 1 的下标)

`__builtin_ctz()` // 从最低位开始连续有多少个 0 (即, 最低位 1 的下标)

`__builtin_clz()` // 固定宽度中, 前导 0 个数 (`32 - __builtin_clz()` - 1 即为最高位 1 的下标)

`__builtin_popcount()` // 转化成二进制之后, 1 的个数

2. 枚举一个集合的所有子集 (包括空集)

```
for (int t = s; ; t = (t - 1) & s) {  
    // ...  
    // 处理这些子集合  
    // ...  
    if (!t) break;  
}
```

3. 构造区间 $[l, r)$ 的掩码

`int x = ((1 << (r - l)) - 1) << l; eg : l = 3, r = 6, 则 s = 111000`

4. 使用掩码修改集合

`t = t & x` 只保留 $[l, r)$ 的元素

`t = t | x` 不重复的加入 $[l, r)$ 的元素

`t = t | ~x` 不重复的加入 $[l, r)$ 以外的元素 (这里的“以外”是相对于所有的32位, 而不仅仅是有效集合范围内)

`t = t & ~x` 只删去 $[l, r)$ 的元素

5. 单个元素的增删查改

`uint64_t bit = uint64_t{1} << k; // 元素 k 对应掩码`

`t |= bit; // 加入 k`

`t &= ~bit; // 删除 k`

`t ^= bit; // 切换 k: 存在则删除, 不存在则加入`

`bool has = t & bit; // 查询 k 是否存在`

// 设置某一位为特定值

`t = (t & ~bit) | (uint64_t(value) << k); // value 为 0 或 1`

6. lowbit 与最低位 1

`uint64_t lowbit = x & -x;`

// 取出最低位的 1

`x = 10110000`

`-x = 01010000 // 补码意义下观察低位`

`x & -x = 00010000`

```
// 常见操作
x &= x - 1; // 删除最低位的 1
x ^= x & -x; // 同样删除最低位的 1

// 手动统计 1 (Fenwick 树中 lowbit 的基础)
int count = 0;
while (x) {
    x &= x - 1;
    ++count;
}
```

7. 删除低 k 位

`t = (t >> k) << k` 删去 `[0, k)` 的元素

8. `~u` 判断 `-1` (最短解中常见, 要能看懂, 原理是补码)

```
int u;
cin >> u;
if (~u)    <=>  if (u != -1)
    if (!~u) <=>  if (u == -1)
```

9. 常见判断技巧

```
// 判断 x 是否是 2 的幂, 因为 2 的幂只有一个 1
bool isPowerOfTwo = x > 0 && (x & (x - 1)) == 0;
```

// 判断集合 a 是否为集合 b 的子集

```
bool isSubset = (a & b) == a;

uint64_t intersection = a & b; // 交集
uint64_t unionSet      = a | b; // 并集
uint64_t difference    = a & ~b; // 差集 a - b
uint64_t symmetric      = a ^ b; // 对称差

unsigned value = (x >> k) & 1u; // 取第 k 位
```

10. 优先级和类型陷阱

```
// 构造掩码时不要使用普通的 1
1 << 40 // 通常错误: 1 是 32 位 int
uint64_t{1} << 40 // 正确
1ULL << 40 // 正确
```